

# / “”...”	commenti
...	non devo scrivere niente io
pass	devo scrivere qualcosa

METODI (def):

	Metodo di istanza: (self)
@classmethod	Metodo di classe (cls)
@staticmethod	Metodo statico

LETTURA DA FILE:

```
f = open("input.txt", "r").read().splitlines()
```

FOR RISTRETTO:

```
categorie = [c.categoria for c in lista_clienti]
```

TRICK:

if nomeLista: (lista non vuota, con il *not* lista vuota)

AGGIUNTE:

<i>geopy.</i>	<i>distance.distance((x, y), (x, y)).km</i>	(distanza tra 2 coordinate)
---------------	---	-----------------------------

<i>itertools.</i>	<i>combinations(lista, n)</i>	(combinazioni / permutazioni
	<i>permutations(lista, n)</i>	n elementi della lista)

VARIABILE PRIVATA: `_nomeVariabile`

Getter:

`@property`

`def nomeVariabile(self):`

Setter:

`@nomeVariabile.setter`

`def nomeVariabile(self, value):`

DUNDER METHODS:

`__str__` : da leggere (basta stampare l'oggetto)

`__repr__` : chiamata al costruttore (in formato stringa, si vede in debug)

`__eq__` : istanze uguali (==) *other: "Classe"*

`__lt__` : comparatore (<, usata nel sort()), sempre insieme a `__eq__` *other: "Classe"*

`__hash__` : NECESSARIO se classe usata come chiave dizionario o in un set. Insieme a `__eq__`

INHERITANCE (Gerarchia):

`class Figlia(Padre1, Padre2):` *(Padre1.__init__(...) per ogni padre)*

DATACLASS: `! import !`

`@dataclass(eq=False, order=True)`

`class Classe:`

`attr1: list[Classe]`

`attr2: int = field(compare=False)`

`def funzione():`

IMPLEMENTA di base :

`__init__`

`__repr__`

`__eq__` (campi tutti uguali)

PROTOCOLS:

`from typing import Protocol`

`class HaFunzione(Protocol):`

`def funzione(self) -> str:`

`...`

`list[HaFunzione]` -> solo chi ha quella funzio.

VERIFICA CHE UNA CLASSE ABBIA UNA

CERTA FUNZIONE. OPPURE USARE:

`isinstance(oggetto, Classe)`

MODULI:

`import nomeModulo as nm`

`if __name__ == "__main__":`

SORT:

nomeLista.sort(key = lambda x: x.attr, reverse=True) (key se ordinamento diverso da `__lt__`)

INTERFACCIA (MVC):

TextField: *self._view.nomeTF.value* (leggi valore)

self._view.nomeTF.disabled = True (disabilita)

DropDown: *self._view.nomeDD.value* (leggi valore)

ListView: *self._view.nomeLV.controls.append(ft.Text(f"", color=""))*

self._view.nomeLV.controls.clear() (svuota)

ElevatedButton: *self._view.nomeEB.disabled = True* (disabilita)

DATABASE (DAO):

SELECT (ritorna lista vuota se nessun elemento)

GRAFI (NODI sempre hashable): <https://networkx.org/>

import networkx as nx

g = nx.Graph() (indiretto)

nx.DiGraph() (diretto)

nx.MultiGraph() (indiretto con archi multipli)

nx.MultiDiGraph() (diretto con archi multipli)

g.add_edge(nodo1, nodo2, weight=...) (aggiungi arco con eventuali parametr)

function=...

add_node(nodo) (aggiungi nodo)

add_edges_from(lista) (aggiungi tanti archi)

add_weighted_edges_from(lista) (aggiungi tanti archi pesati)

add_nodes_from(lista) (aggiungi tanti nodi)

nodes() (restituisce la lista dei nodi)

<code>edges(data=True)</code>	(restituisce la lista degli archi formato: [0] arco partenza [1] arco arrivo [2][“weight”] peso)
<code>get_edge_data(nodo1, nodo2)</code>	(restituisce il peso dell’arco)
<code>successors(nodo)</code>	(successori del nodo se grafo diretto)
<code>predecessors(nodo)</code>	(predecessori del nodo se grafo diretto)
<code>has_edge(nodo1, nodo2)</code>	(True se esiste l’arco)
<code>neighbors(nodo)</code>	(Nodi adiacenti a uno (vicini))
<code>remove_edge(nodo1, nodo2)</code>	
<code>remove_node(nodo)</code>	
<code>clear()</code>	
<code>clear_edges()</code>	
<code>in/out_degree(nodo)</code>	(Grado del nodo)
<code>subgraph(nodi).copy()</code>	(Sottografo con solo i nodi nella lista)
<code>get_edge_data(nodo1, nodo2)[“weight”]</code>	(Dati dell’arco)

DiGraph

<code>.in_edges(nodo, data=True)</code>	(LISTA di archi entranti nel nodo)
<code>.out_edges(nodo, data=True)</code>	(LISTA di archi uscenti dal nodo)

<i>nx.</i> <code>connected_components(grafo)</code>	(USARE LIST() DAVANTI per avere la lista di tutte le com. connesse del grafo. Ognuna è una lista di nodi)
<code>number_connected_components(grafo)</code>	
<code>node_connected_component(grafo, nodo)</code>	(componente connessa contenente “nodo”)
<code>shortest_path(grafo, source, target)</code>	(cammino minimo con DJ o BF)
<code>has_path(grafo, source, target)</code>	(True se esiste percorso)

weakly_connected_components(grafo)

(gruppi di nodi in un grafo in cui ogni nodo è raggiungibile da qualsiasi altro, ignorando la direzione)

g[nodo] (dizionario con chiave i nodi adiacenti e valori i parametri come il peso)

g[nodo1][nodo2][“weight”] (attributi dell’arco)

nodo in g (True se ‘nodo’ è un nodo di g)

for n in g: (ciclo sui nodi)

for nbr in g[nodo]: (ciclo sui nodi adiacente a ‘nodo’)

for n in predecessors(nodo) / successors(nodo)

union(g1,g2) (unione dei grafi)

disjoint_union(g1,g2) (unione dei grafi con tutti i nodi diversi)

compose(g1,g2) (grafo con i nodi in comune tra i due)

complement(g) (grafo con nodi uguali ma archi solo dove non ci sono in g)

create_empty_copy(g) (copia vuota del grafo)

convert_to_undirected(g) (conversione in indiretto)

convert_to_directed(g) (conversione in diretto)

NODI RAGGIUNGIBILI DA UN NODO SORGENTE:

<https://networkx.org/documentation/stable/reference/algorithms/traversal.html>

(entrambi trovano gli stessi nodi con ordine diverso)

AMPIEZZA – BFV

(in ordine di distanza – CAMMINO MINIMO SE NON PESATO)

nx. bfs_edges(g, sorgente) (restituisce gli archi)

bfs_tree(g, sorgente) (restituisce l’albero)

bfs_layers(g, sorgente) (restituisce i livelli)

STESSI ATTRIBUTI PER ENTRAMBI

PROFONDITA' – DFV (cammini più lunghi possibili – COMPONENTE CONNESSA)

nx. dfs_edges(g, sorgente) (restituisce gli archi connessi)

dfs_tree(g, sorgente) (restituisce l'albero dei nodi connessi)

node_connected_component(g, source) (restituisce la lista dei nodi connessi)

dfs_predecessors(g, source) (restituisce un DIZIONARIO con il predecessore di ognuno)

dfs_successors(self._grafo, nodo).values() **(cammino + lungo da 'nodo') ![0]!**

CAMMINO MINIMO: ! se non pesato usare BFV !

https://networkx.org/documentation/stable/reference/algorithms/shortest_paths.html

FLOYD-WARSHALL (da tutti i nodi verso tutti i nodi – NO CICLI NEGATIVI)

fw = nx.floyd_warshall(g, weight="weight")

results = {a: dict(b) for a,b in fw.items} (dizionario con chiave sorg. e dest.)

BELLMAN-FORD (da un nodo a tutti gli altri – RILEVA CICLI NEGATIVI)

bf = all_pairs_bellman_ford_path(g, weight="weight") (percorso tra tutti i nodi)

DIJKSTRA (da un nodo a tutti gli altri – DIRETTI O NO – NO PESI NEGATIVI)

d = dijkstra_path(g, source, target, weight="weight") (percorso da source a target, risultato come lista dei nodi percorsi)

d = single_source_dijkstra(g, source, weight="weight") (da source a tutti gli altri – LUNGHEZZA E PERCORSO)

len = dijkstra_path_length(G, source, target, weight="weight") (costo percorso)

CICLI:

si dice grafo ... se contiene almeno un ciclo ...

EULERIANO (tutti gli archi 1 volta sola, “percorso” se source != target)

<https://networkx.org/documentation/stable/reference/algorithms/euler.html>

(ciclo - OGNI NODO HA GRADO PARI)

(percorso – MAX 2 NODI CON GRADO DISPARI)

eulerize() (rende grafo euleriano)

HAMILTONIANO (tutti i nodi 1 volta sola, “percorso” se source != target)

https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithms.tournament.hamiltonian_path.html

h = hamiltonian_path(g) (ritorna lista di nodi del percorso)

TRAVELING SALESMAN (CICLO HAMILTONIANO)

https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithms.approximation.traveling_salesman.christofides.html#networkx.algorithms.approximation.traveling_salesman.christofides

SQL:

tab1 LEFT JOIN tab2 ON ... = (null dove non presente valore)

BETWEEN ... AND ...

REPLACE(string, old_string, new_string)

LIKE (a%) (iniziano con ‘a’)

CAST(value AS type)

ORDER BY ... ASC/DESC

COALESCE(valore, 0) (restituisce primo valore non null)

Per creare grafi orientati si può fare il join e controllare che i valori siano diversi (no < o >) e poi mettere la disuguaglianza stretta sul valore del peso dell’arco.

Il controllo delle date può essere fatto come stringhe

! fare sottografo per operazioni su componente connessa !

- Un cammino è **semplice** se al suo interno **non si ripetono gli archi** (o spigoli).
- Un cammino è **elementare** se al suo interno **non si ripetono i nodi** (o vertici).

Se ELEMENTARE, allora anche SEMPLICE

ACICLICO – non 2 volte lo stesso vertice nel percorso